

《决策原生方法论》

《决策原生方法论》

Decision-Native Methodology

第一部分 | 决策原生 (Decision-Native)

决策原生是一种用于构建组织级决策机制的范式与方法论。

它源自一个长期存在但未被系统性解决的问题：在高风险与不确定性条件下，关键决策反复发生。这些决策一旦被组织采纳，便会产生不可逆的结果。然而，在多数情况下，决策的形成过程、依据来源与责任归属无法被稳定记录与回放，决策只能依赖个体经验或临时形成的集体判断完成，难以沉淀为组织可持续使用的决策能力。

这一问题的根源在于，决策缺乏可被组织承载的机制形态。当决策无法明确其发生位置、依据来源与责任主体时，组织只能依赖个体或临时集体承担后果，而无法形成可运行、可演进的决策机制。

决策原生正是在此基础上构建的一种范式。该范式以“决策”作为核心对象，将决策视为一种需要被组织正式采纳并承担后果的行为，并据此重新界定决策机制的设计方式。

第一性约束用于界定决策机制是否具备成立的前提。第一性约束由可审计性与可追责性构成。只有当决策过程能够被审计、决策结果能够被追责时，该决策才具备被组织采纳的必要条件。

在决策机制中，构件是核心概念的承载形式，用于将核心概念落实为可运行的机制。决策原生方法论由第一性约束与一组相互依赖的核心概念构成，用于支撑决策机制的成立、运行与持续演进。当决策机制由核心构件组织并持续运行时，其整体形态表现为组织级决策基础设施，为需要明确责任主体、可审计过程与可追责结果的决策场景提供统一的承载方式。

为避免概念混用与层级错位，后续内容将严格区分范式、核心概念、构件与关键术语的不同角色与边界。

基于上述范式与方法论，后续章节将依次展开核心概念、构件关系与关键术语，用以说明决策机制如何成立、如何运行。

第二部分 | 核心概念 (Core Concepts)

核心概念是构成决策原生方法论不可替代的最小集合。

核心概念的判定遵循以下原则：

- 最小性：缺少任一核心概念，决策原生方法论即无法成立；
- 依赖性：核心概念之间存在不可跳过的依赖关系；
- 约束性：核心概念对决策机制的运行形成必要约束。

基于上述判定原则，决策原生方法论的核心概念包括：第一性约束、决策机制、决策图谱、决策节点、强制 HITL、业务对象、知识单元、Skill、决策账本、决策轨迹与失效模式。

在高风险与不确定性条件下，所有核心概念共同支撑决策机制的成立，并具备可审计、可追责与可演进的运行前提。核心概念回答的是“决策原生方法论是否成立”以及“决策机制是否成立”。

2.1 第一性约束(First-Order Constraints)

第一性约束用于判定决策机制是否具备成立的必要条件。

在决策原生方法论中，第一性约束具体由两项条件构成：**可审计性与可追责性**。即，决策过程必须可被审计，且决策结果必须可被追责。它们同时用于回答：**某一决策机制是否具备成立的前提**。

其中：

- 可审计性（Auditability）：一次决策在形成与采纳过程中，其输入依据、引用规则、路径选择与采纳记录能够被结构化留存，使决策在事后可被回放、核验与合规检查。
- 可追责性（Accountability）：一次决策被采纳后，其结果能够在责任结构上明确对应到被授权的责任主体或责任链条，使责任可被定位、追溯并实施追责。

在高风险与不确定性条件下，同时满足上述条件的决策机制，是决策结果被组织采纳的必要前提。

2.2 决策机制（Decision Mechanism）

决策机制是指在高风险与不确定性长期存在、关键决策反复发生的条件下，使决策能够在组织级被正式采纳并可承担责任的**组织级基础设施**。

在决策原生方法论中，决策机制用于将第一性约束中所要求的可审计性与可追责性，落实为一种可被组织采纳并能够持续运行的决策机制形态。

决策机制由一组相互依赖的核心构件共同构成，包括决策图谱、决策节点、强制 HITL、决策账本、决策轨迹与失效模式。这些构件在明确的依赖关系、约束条件与责任关系下被组织起来，共同形成支撑决策机制成立、运行与持续演进的**组织级决策基础设施**。

其中，决策图谱用于表达决策机制的总体结构与路径关系；决策节点用于界定判断发生的位置及其责任落点；决策账本用于形成可审计的最小承载单元；决策轨迹与失效模式用于支持决策结果的追溯与演进；强制 HITL 用于确保在关键节点上责任主体不可被技术系统替代。

第一性约束回答“决策机制是否成立”，决策机制回答“如何在组织级使决策成立”。

2.3 决策图谱（Decision Graph）

决策图谱是决策机制的结构化表达，用于呈现决策机制在一次完整决策中所采用的节点组织方式与路径关系。

在决策原生方法论中，决策机制需要具备可运行的结构，才能支撑可审计、可追责的决策机制运行。决策图谱通过对**决策节点及其路径关系的明确描述**，使决策机制在结构上具备被审计与被追责的前提。

决策图谱用于描述一次完整决策在机制层面的展开方式：哪些决策节点被纳入决策过程，节点之间如何形成前置与分支关系，以及决策路径如何从入口节点展开并走向完成。该表达为责任主体定位、审计回放以及决策机制的后续调整提供明确依据。

在决策机制中，决策图谱与其他核心构件形成明确的配合关系：

决策图谱定义决策节点之间的结构位置与路径关系，决策节点在图谱所确定的位置上承担具体判断与责任主体；强制 HITL 以节点约束的形式嵌入图谱结构中，用于确保关键节点上的责任不可被技术系统替代；决策账本依据决策图谱所确定的节点结构，对各节点结果进行记录；决策轨迹沿决策图谱所定义的路径，对一次完整决策的过程进行串联；失效模式以决策图谱所呈现的结构为基础，对失败发生的位置与原因进行定位与分析。

决策图谱回答“**一次完整决策在结构上如何被组织与展开**”。

2.4 决策节点（Decision Node）

决策节点是决策图谱中的最小责任单元。

在决策原生方法论中，决策节点用于标明一次判断在决策机制中发生的位置，使判断责任能够被明确定义并追溯。每一个决策节点对应一次可被组织采纳的判断，其结果必须能够被记录、审计与追责。

每一个决策节点表示一次判断的发生。该判断并不等同于技术执行或自动推理，而是以责任可定位为前提。任何无法明确责任主体的判断，不构成决策节点。

在结构上，每一个决策节点必须明确界定以下要素：

判断目标、输入条件、判断依据、输出状态以及对应的责任主体。

其中，判断依据由知识单元提供，节点本身不生成规则，而是在既定规则下完成判断并触发状态变更。

在决策机制中，决策节点与其他核心构件形成明确的分工关系：

决策图谱用于确定节点在整体结构中的位置及其前置与分支关系；

强制 HITL 用于确保特定节点上的判断必须由责任主体介入；

决策账本用于记录节点的判断结果；

决策轨迹用于沿节点路径串联一次完整决策；

失效模式以节点为定位单位，对失败发生的位置进行分析。

决策节点关注的是：责任在何处发生。

决策节点回答的是：“**责任在决策机制中如何被定位**”。

入口决策（Entry Decision）

入口决策是一种用于判断是否启动某一决策路径的决策节点。

在决策原生方法论中，并非所有现实问题都会进入决策机制。入口决策用于在决策机制内部，对是否启动某一决策路径作出判断，从而避免在条件尚不具备时展开高风险或不可逆的决策过程。

作为决策节点的一种，入口决策同样承担明确的责任要求。其判断结果必须写入决策账本，并能够被审计与追责。入口决策判断的对象不是具体业务结论，而是决策路径是否应当被启动。

入口决策的输出通常体现为对业务对象状态的明确标记，例如“未启动 / 已启动”，并形成对应的决策账本快照。该状态写入本身即构成一次可被组织采纳的决策结果。

入口决策不替代后续决策节点的判断，也不产生最终业务结论。其作用在于对决策机制的启动施加约束，使决策机制仅在条件具备时被激活。

在决策图谱中，入口决策作为路径起点，其判断结果直接决定后续节点是否被激活。若入口决策判定不予启动，则对应的决策路径在结构上不被展开。

入口决策回答的是：“是否应当启动这一决策路径”。

2.5 强制 HITL（Enforced Human-in-the-Loop）

强制 HITL 是决策机制中的一种约束性构件，用于在满足预设风险或不确定性条件时，强制要求责任主体介入决策判断过程。

在决策原生方法论中，强制 HITL 并非事后兜底措施，而是在机制层被显式嵌入的判断约束。其触发条件、介入位置与授权边界在决策发生前即被结构化定义，并在决策运行过程中不可被绕过。

强制 HITL 的作用在于防止责任在自动化判断过程中被隐性转移，使关键判断在必要位置由明确的责任主体作出并承担后果。任何被强制 HITL 约束的决策节点，若未完成责任主体介入，其判断结果不得被采纳。

强制 HITL 与决策节点、决策账本形成明确配合关系：责任主体的介入行为被视为判断过程的一部分，其结果必须写入决策账本，并纳入后续审计与追责范围。

强制 HITL 回答的是：“在何种条件下，判断必须由责任主体介入完成”。

2.6 业务对象（Business Object）

业务对象是一次决策所作用的统一承载体，用于结构化承载现实世界中与决策相关的信息。

在决策原生方法论中，业务对象用于统一承载事实、证据与状态，是决策节点进行判断与写入结果的对象基础。决策机制对业务对象的操作，本质上表现为对其状态的判断与写入。

业务对象是决策机制中被判断、被记录并承担后果的对象载体。通过对业务对象的持续记录与状态变更，决策过程得以被审计，责任得以被追溯。

业务对象与其他核心构件形成明确分工关系：Skill 负责从外部输入中抽取事实与证据并写入对象字段；决策节点基于对象中的事实与证据作出判断并更新状态；决策账本记录对象状态变更的结果；决策轨迹以对象为线索串联一次完整决策过程。

业务对象回答的是：“决策所作用并承担后果的对象是什么”。

2.7 知识单元 (Knowledge Unit)

知识单元是决策节点在判断过程中被显式引用的判断依据，用于支撑判断的成立性。

在决策原生方法论中，任何可被组织采纳的判断，都必须能够明确其依据来源。知识单元承担的正是这一职责：将法律法规、行业规范、组织规则或经验证的经验，以可引用、可追溯的形式提供给决策节点。

知识单元本身不执行判断，也不产生状态变更，其作用在于为判断提供明确的依据来源，并约束判断的适用范围。决策节点在作出判断时，必须显式引用相应的知识单元；未能引用知识单元的判断，不具备通过审计的条件。

知识单元具有只读属性，并具备明确的版本标识与适用效力，其内容在决策运行过程中不可被修改。

知识单元回答的是：“这一判断依据来自何处”。

2.8 Skill (能力单元)

Skill 是决策机制中被决策节点调用的最小可执行能力单元，用于完成与判断相关的信息处理操作。

在决策原生方法论中，Skill 的职责在于读取业务对象、执行计算或校验操作，并输出结构化结果。Skill 的输出不得被直接采纳为判断，其结果必须经由决策节点结合知识单元后，方可形成可被采纳的判断结果。

Skill 不承担决策责任，其实现方式可以是模型、规则、人工操作或外部系统。Skill 的核心要求是可替换性与可审计性，而非智能程度。

Skill 与其他核心构件形成明确分工关系：Skill 负责执行具体操作；决策节点负责判断与责任承担；知识单元为判断提供依据；业务对象承载输入与输出结果。

Skill 回答的是：“判断所需的操作由何种能力完成”。

2.9 决策账本 (Decision Ledger)

决策账本是对决策节点判断结果进行记录的结构化记录集合，是审计与追责的最小承载单元。

在决策原生方法论中，决策账本用于将一次决策节点的判断结果在特定时刻固化为可核验的记录。每一条账本记录对应一次决策节点完成状态写入后的结果确认，用于明确记录该判断发生的时间、节点位置、判断依据、状态结果以及对应的责任主体，使该判断具备被审计与追责的成立条件。

决策账本以追加写入 (append-only) 的方式运行。账本记录一经生成，不得被修改或覆盖，仅允许在其后追加新的判断记录。任何能够被组织正式采纳的判断，必须在对应决策节点完成状态写入后形成账本记录；未形成账本记录的判断，不构成可被审计、可被追责的判断结果。

一条最小决策账本记录应至少包含以下要素：

决策节点标识、业务对象标识、判断发生时间、写入的状态结果、所引用的事实与证据标识、对应的责任主体，以及所依据的决策机制或规则版本。

在决策机制中，决策账本与其他核心构件形成明确分工关系：

决策节点在完成判断并写入状态后生成账本记录；

业务对象作为账本记录所指向的判断承载对象；

决策轨迹由多个账本记录按决策图谱路径顺序串联形成；

失效模式以账本记录为最小定位单位，对失败发生的位置与原因进行分析。

决策账本回答“这一判断在何时、何处、基于何种依据被作出，并由谁承担责任”。

2.10 决策轨迹（Decision Trajectory）

决策轨迹是由多个决策账本记录按照决策图谱所定义的路径顺序组织而成的一次完整决策过程的结构化记录，用于描述某一业务对象在一次决策机制运行中实际经历的节点路径与状态变更序列。

决策轨迹由决策账本记录构成，不引入新的判断，也不改变既有判断结果，其内容仅来源于已发生并已记录的决策节点判断行为。

在决策机制中，决策轨迹与其他核心构件形成如下关系：

决策账本作为最小记录单元，提供单个决策节点的判断结果；

决策图谱定义决策轨迹可能展开的结构边界与路径约束；

决策轨迹在账本记录之上组织形成一次完整决策的过程实例；

失效模式以决策轨迹为分析对象，对失败发生的路径位置与结构原因进行定位。

决策轨迹是决策机制进行回放、审计与演进调整的基础承载对象。

决策轨迹回答的是：“一次完整决策是如何沿着决策图谱被实际执行并形成结果的”。

2.11 失效模式（Failure Pattern）

失效模式是对决策机制在运行过程中出现的失效形态所作的结构化抽象，指在多个完整决策轨迹中反复出现、并导致决策机制无法按预期成立的结构性条件组合。

在决策原生方法论中，失效模式并不针对单一案例或具体判断结果，而是基于多条决策轨迹的对比与归纳，对失效在结构层面的共性特征进行提炼。只有当某一失效形态在不同业务对象、不同决策轨迹中重复出现时，才构成可被纳入方法论的失效模式。

失效模式用于界定决策机制的有效边界。通过明确在何种结构条件下决策机制无法继续成立，失效模式为决策图谱的结构调整、决策节点的重构以及知识单元的修订提供依据。

在决策机制中，失效模式与其他核心构件形成明确分工关系：

决策轨迹提供失效分析的对象基础；

决策账本作为失效定位的最小记录单元；

失效模式在轨迹层面识别重复出现的结构性失效路径；

决策图谱依据失效模式对结构进行调整或阻断；

知识单元依据失效模式补充或修订判断依据。

失效模式并不直接参与单次决策的判断过程，其作用在于为决策机制的持续修订与结构演进提供稳定依据。

失效模式回答“在何种结构条件下，决策机制无法继续成立”。

第三部分 | 构件关系（Component Relationships）

本部分用于描述《决策原生方法论》中各核心构件之间的制度性关系。

构件关系不描述实现方式、技术流程或执行顺序，仅用于说明在决策机制成立、运行与演进过程中，各构件在责任、约束、承载与反馈层面如何相互关联。

构件关系以第二部分“核心概念”为唯一依据，其作用在于：

- 明确各核心构件在决策机制中的依赖与约束关系；
- 说明判断、记录、分析与反馈在制度层面的衔接方式；
- 为构件关系图生成、业务关系展开及后续决策机制的结构化实现提供稳定、可复用的结构基础。

构件关系本身不构成流程，不描述执行细节。

3.1 构件关系布局规则（Layout Rules）

构件关系图遵循以下布局规则：

- 决策机制**作为组织级决策基础设施，构成构件关系图的最外层边界，所有构件均位于其范围之内。
- 决策图谱**位于决策机制内部，用于承载决策节点之间的结构关系。
- 决策节点**仅存在于决策图谱之中，是判断发生与责任承担的位置。
- 入口决策**作为决策节点的一种类型，仅出现在决策图谱的起始位置，用于激活后续决策节点。
- 强制 HITL**不作为独立构件存在，而以约束关系的形式作用于特定决策节点。
- 业务对象**作为统一承载体，位于决策节点的可读写范围内，用于承载事实、证据与状态。
- 知识单元**与 **Skill**不位于决策图谱内部，而以被引用、被调用的关系连接至决策节点。
- 决策账本**位于决策节点之后，用于承载节点判断结果的不可变记录。
- 决策轨迹**由多条决策账本记录构成，用于表达一次完整决策的过程实例。
- 失效模式**位于分析与反馈侧，不直接参与决策运行，仅通过反馈关系作用于结构与规则修订。

3.2 构件关系定义表（Relation Dictionary）

下表定义核心构件之间允许存在的制度性关系。未列出的关系在方法论中不成立。

主体构件	关系名称	客体构件
决策机制	包含	决策图谱
决策机制	包含	决策账本
决策机制	包含	决策轨迹
决策机制	包含	失效模式
决策图谱	包含	决策节点
决策图谱	定义入口	入口决策
入口决策	激活	决策节点
决策节点	读取	业务对象
决策节点	引用	知识单元
决策节点	调用	Skill
决策节点	写入	业务对象状态
决策节点	生成	决策账本
强制 HITL	约束	决策节点
决策账本	组成	决策轨迹
决策轨迹	抽象为	失效模式
失效模式	反馈至	决策图谱
失效模式	反馈至	决策节点
失效模式	反馈至	知识单元
业务对象	包含	事实
业务对象	包含	证据
业务对象	包含	状态

3.3 构件关系释义（Relation Glossary）

以下释义用于说明各关系在决策原生方法论中的制度含义，所有释义均与核心概念保持一致。

包含

表示某构件在制度上作为另一构件的组成部分存在，用于界定决策机制的边界。

定义入口

表示在结构上明确决策图谱的进入位置，用于限定哪些决策路径可以被启动。

激活

入口决策用于判定业务对象是否具备进入后续决策节点的条件，其结果决定是否进入决策图谱中的首个决策节点。

读取

表示在判断过程中从业务对象中获取事实与证据，不改变其状态。

引用

表示在判断过程中显式使用制度性依据，作为判断成立的前提条件。

调用

表示使用可执行能力完成信息处理，其输出不直接构成决策结果。

写入

表示将判断结果写入业务对象状态，是决策成立的关键制度动作。

生成

表示在状态写入后形成对应的制度性记录，用于审计与追责。

约束

表示对判断或状态写入施加条件性限制，要求责任主体介入确认。

组成

表示多个不可变记录在制度上构成一次完整决策过程。

抽象为

表示从多个决策轨迹中提炼重复出现的结构性失效形态。

反馈至

表示将分析结果反向作用于结构设计、节点重构或知识单元修订，而不参与当前决策运行。

包含（业务对象内部）

表示业务对象对事实、证据与状态的统一承载关系。

3.4 构件关系图

以下为构件关系图提示词：

Draw a "Methodology-Level Component Relationship Diagram" for the "Decision-Native Methodology".

1. Visual Style & Layout:

- **Style:** Professional Enterprise Architecture Diagram. High contrast, clean, minimalist.
- **Line Style:** Use **straight orthogonal lines (right angles)** for all forward process flows. Use **curved lines ONLY** for backward feedback loops.
- **Language:** All labels must be in **Chinese** (Simplified).

- **Container:** The entire diagram is enclosed in a large rounded boundary labeled "**决策机制** (组织级决策基础设施)".

2. Component Containers (Clusters):

- **Cluster A: "决策图谱" (Decision Graph)**
 - Located in the center.
 - Contains two nodes: "**入口决策**" (Entry Decision) and "**决策节点**" (Decision Node).
 - Attach a "**强制 HITL**" label (as a lock constraint icon) directly to the "**决策节点**".
- **Cluster B: "业务对象" (Business Object)**
 - Located below or to the left of the Decision Graph.
 - Contains three sub-blocks stacked vertically: "**事实**" (Facts), "**证据**" (Evidence), "**状态**" (State).
- **Cluster C: "决策轨迹" (Decision Trajectory)**
 - Located to the right of the Decision Graph.
 - Contains multiple "**决策账本**" (Decision Ledger) items arranged sequentially.

3. Independent Components:

- "**知识单元**" (Knowledge Unit): Located above the Decision Graph.
- "**Skill**": Located above the Decision Graph (next to Knowledge Unit).
- "**失效模式**" (Failure Pattern): Located at the far right end.

4. Relationship Logic & Arrow Directions (Strict):

[Forward Flow]

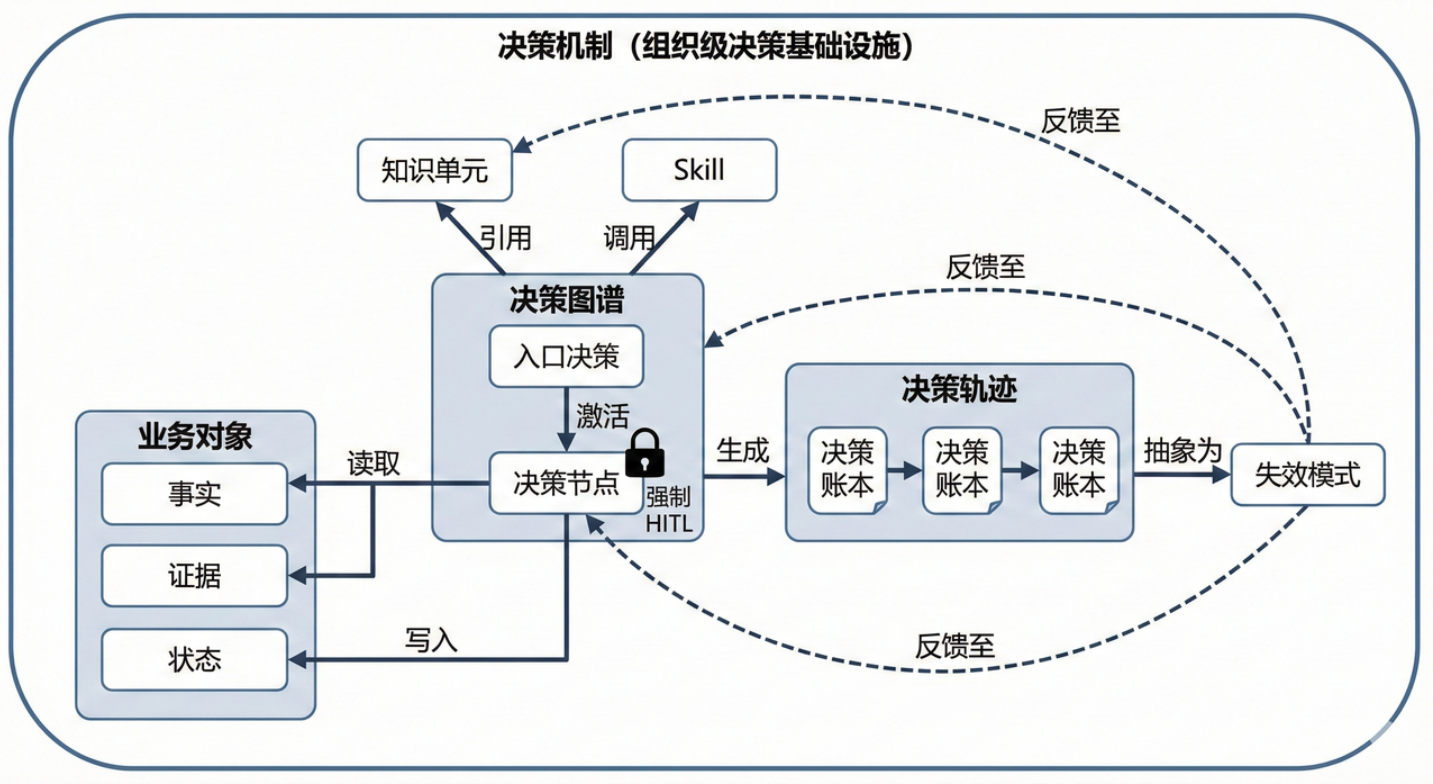
- **入口决策** → (Arrow: **激活**) → **决策节点**
- **决策节点** → (Arrow: **读取**) → **事实** (Inside Business Object)
- **决策节点** → (Arrow: **读取**) → **证据** (Inside Business Object)
- **决策节点** → (Arrow: **写入**) → **状态** (Inside Business Object)
- **决策节点** → (Arrow: **引用**) → **知识单元**
- **决策节点** → (Arrow: **调用**) → **Skill**
- **决策节点** → (Arrow: **生成**) → **决策账本**
- **决策账本** → (Arrow: **组成**) → **决策轨迹**
- **决策轨迹** → (Arrow: **抽象为**) → **失效模式**

[Feedback Loops] (Use curved dashed lines)

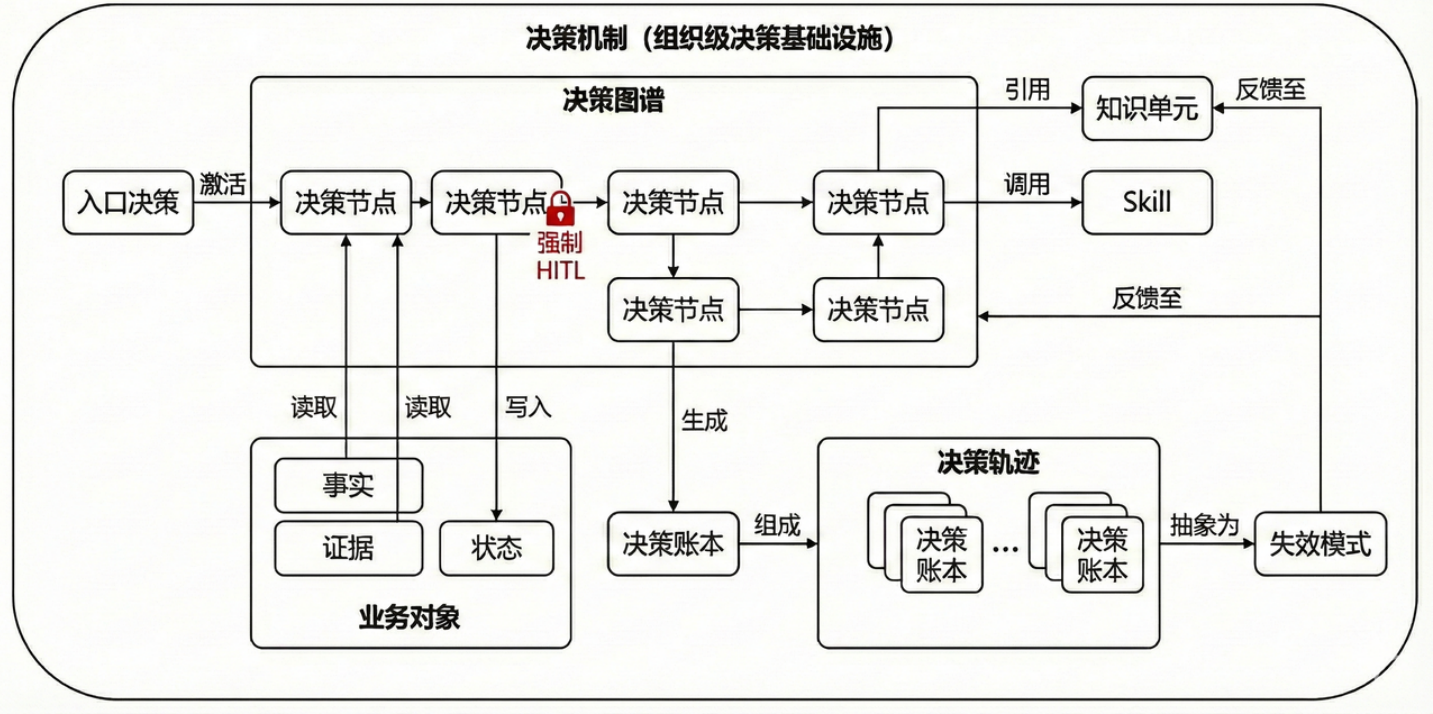
- **失效模式** → (Arrow: **反馈至**) → **决策图谱**

- 失效模式 → (Arrow: 反馈至) → 决策节点
- 失效模式 → (Arrow: 反馈至) → 知识单元

经过提示词所生成的构件关系图效果如下：



决策原生方法论：构件关系图（Decision-Native Methodology: Component Relationship Diagram）



第四部分 | 关键术语 (Key Terminology)

本部分用于界定《决策原生方法论》中用于**精确指代、避免歧义并支撑实现与讨论**的关键术语。

关键术语服务于核心概念，但**不构成方法论骨架**。

核心概念用于回答“方法论是否成立”，而关键术语用于回答“如何准确地讨论与实现已成立的方法论”。

术语是可被调用、被使用、被实现、被替换的命名单元，其作用在于：

- 保证设计、实现与讨论中的指代一致性；
- 避免将实现细节误写为方法论前提；
- 为构件关系表达、业务关系映射与后续系统实现提供稳定语义基础。

4.1 决策 (Decision)

决策是被组织采纳、写入业务对象状态并承担责任的**制度性结果**。

在决策原生方法论中，决策不同于判断、结论或建议：

- 判断是发生在决策节点内部的认定行为；
- 决策是该判断被正式采纳并写入状态后的制度性成立状态。

只有在满足第一性约束（可审计、可追责）的前提下，判断才可能转化为决策。

4.2 判断 (Judgment)

判断是发生在决策节点内部的认定行为，可由人或 Agent 产生，其本身不必然产生制度后果。

判断仅在被采纳、写入业务对象状态并进入决策账本后，才形成决策。

判断是强制 HITL 约束条件的重要来源，但不等同于责任承担。

【待确认】是否将“判断”作为关键术语长期保留，或仅作为解释性用语。

4.3 采纳 (Adoption)

采纳是将节点判断转化为组织决策的制度性动作。

采纳意味着：

- 责任主体在授权边界内确认该判断进入决策机制；
- 判断结果被写入业务对象状态；
- 相应记录进入决策账本并承担制度后果。

采纳是判断与决策之间的制度分界点。

4.4 可审计（Auditability）

可审计指决策过程具备可回放的事实基础，使审计方能够追溯：

“谁在什么条件下、基于哪些证据、按哪一版机制作出何种判断与采纳”。

可审计性是可追责性的前提条件，二者不可分离。

4.5 可追责（Accountability）

可追责指决策结果在制度上具有明确责任归属。

当决策被采纳并产生后果时，责任必须能够稳定定位到授权边界内的责任主体；

若无法追责，审计将失去制度意义。

4.6 可演进（Evolvability）

可演进指决策机制在保持第一性约束不变的前提下，具备进行制度性更新的能力。

可演进的对象包括但不限于：

- 风险阈值的调整；
- 强制 HITL 触发条件的更新；
- 决策路径结构的重构；
- 知识单元内容的修订。

并非所有构件均可演进；第一性约束本身不可演进。

4.7 授权边界（Authorization Boundary）

授权边界用于界定：

- 哪些角色在何种条件下拥有作出判断或采纳的授权；
- 该授权如何在决策发生前被冻结、引用与审计。

授权边界与强制 HITL、决策账本中的责任主体字段直接相关。

4.8 黑箱对用户 / 白箱可审计

(Black-box to User / White-box for Audit)

该原则指：

- 对外仅呈现经过采纳的最终决策结果；
- 对内与审计方保留完整、可回放的决策结构与账本记录。

其目的是在用户体验、模型隔离与制度可信度之间取得平衡。

4.9 状态写入 (State Write)

状态写入是决策成立的关键制度动作。

状态写入指将被采纳的判断结果写入业务对象状态，并由此：

- 决定后续决策路径；
- 触发或终止强制 HITL；
- 形成可审计、可追责的制度记录。

状态写入只能由决策节点完成，不能由 Skill 或其他执行单元完成。

4.10 责任主体 (Accountability Owner)

责任主体是决策账本中用于定位责任承担者的制度字段。

责任主体用于回答：

“谁在什么授权边界下，对该节点的判断与采纳承担责任”。

(英文统一采用 **Accountability Owner**)

4.11 回放 (Replay)

回放指基于决策账本与决策轨迹，对一次已完成决策进行结构化重现的能力。

回放用于审计、复盘与失效分析，不参与当前决策运行。

【说明】本术语不等同于核心概念，不引入额外机制，仅作为审计与分析能力的命名约定。

4.12 风险阈值 (Risk Threshold)

风险阈值用于界定在何种不确定性或风险水平下，必须触发强制 HITL 或更高等级的责任介入。

风险阈值本身不构成判断结果，但直接影响决策路径与责任分配。

术语使用说明

- 本部分术语仅用于指代与说明，不改变核心概念含义；
- 未在本表中定义的词语，不得作为制度性结论词使用；
- 若术语与外部行业用语存在歧义，应以本方法论定义为准。

第五部分 | 业务关系 (Business Relationships)

本部分用于描述《决策原生方法论》中，决策机制在真实业务语境中的**抽象级业务关系形态**。

业务关系不描述具体行业流程、系统实现方式或组织分工，而用于说明：

- 决策原生方法论在业务层面**如何被正确使用**；
- 各核心构件在业务运行中**分别承担何种职责与位置**；
- 决策机制在不同业务场景中**适用与不适用的边界条件**。

业务关系以第二部分“核心概念”与第三部分“构件关系”为唯一依据，其作用在于为后续的业务映射、应用指引与工程化落地提供稳定、可复用的抽象基础。

5.1 业务关系的抽象原则

业务关系的抽象遵循以下原则：

(1) 不引入新概念原则

本部分不引入新的方法论概念或构件，仅对已冻结的核心概念与构件关系进行业务语境下的映射说明。

(2) 职责而非流程原则

业务关系描述的是各构件在业务中承担的**职责位置**，而不描述业务流程、执行顺序或操作步骤。

(3) 位置而非实现原则

业务关系强调构件在业务运行中的**存在位置与作用边界**，不涉及具体技术实现、系统架构或组织形态。

(4) 边界可判定原则

所有业务关系描述，必须支持判断某一业务场景是否适合采用决策原生方法论，或应当被明确排除。

5.2 业务关系的层次结构（抽象级）

在业务语境中，决策原生方法论中的构件与关系可被映射为以下抽象层次结构。

该层次结构用于说明构件在业务中的**存在位置**，而非系统分层、流程阶段或组织划分。

(1) 接入层 (Access Layer)

接入层指业务对象进入决策机制之前，与外部业务世界发生接触的位置。

- 该层负责承载业务对象的进入、触发与初始识别；
 - 入口决策在该层发生，用于判定业务对象是否具备进入决策机制的条件；
 - 本层不产生判断结果，不承担责任，仅承担进入条件的制度性界定。
-

(2) 治理层 (Governance Layer)

治理层用于承载与责任、授权与合规相关的业务关系。

- 责任主体、授权边界与采纳行为在该层具有明确业务含义；
 - 强制 HITL 在业务上体现为治理约束，而非技术补丁；
 - 本层确保决策结果在业务层面具备可追责性与合法性基础。
-

(3) 机制层 (Mechanism Layer)

机制层是业务判断实际发生的位置。

- 决策图谱、决策节点、知识单元与 Skill 在该层协同工作；
 - 判断在该层发生，并受第一性约束与构件关系的共同约束；
 - 本层是业务判断逻辑被制度化、可审计表达的核心位置。
-

(4) 运行结果层 (Outcome Layer)

运行结果层用于承载决策运行后的业务结果与过程连续性。

- 决策账本在业务上体现为不可变的判断记录；
 - 决策轨迹在业务上体现为一次完整决策的过程实例；
 - 本层支撑业务复盘、审计回放与过程核验。
-

(5) 分析与演进层 (Analysis & Evolution Layer)

分析与演进层用于承载业务运行后的分析与机制改进输入。

- 失效模式在该层形成，用于抽象重复出现的业务性失效结构；
- 该层不参与当前业务运行；
- 其输出仅以反馈形式作用于决策图谱、决策节点与知识单元的修订。

5.3 典型业务形态（抽象级）

在抽象层面，适合采用决策原生方法论的业务形态通常具备以下特征：

- 决策一旦被采纳，将产生明确且不可逆的组织后果；
- 决策过程需要被审计、回放或在事后接受问责；
- 业务风险无法完全通过规则或模型消除；
- 决策质量依赖责任主体，而非单纯技术输出；
- 决策机制需要在长期运行中持续修订与演进。

上述特征用于判断业务是否具备引入决策原生方法论的必要性，而非用于描述具体行业或业务类型。

5.4 使用边界与禁止性说明

以下业务情形不适合采用决策原生方法论：

- 仅涉及即时计算、无责任后果的自动化任务；
- 决策结果无需被组织采纳或承担责任；
- 决策逻辑完全稳定、无需审计与复盘；
- 业务风险可被完全外包或转移，无需内部承担。

当业务场景长期处于上述状态时，引入决策原生方法论将导致机制冗余或治理成本过高。

5.5 本部分与后续应用的关系

本部分所定义的业务关系，为以下工作提供抽象基准：

- 业务关系映射模板的生成；
- 决策原生方法论在咨询与业务设计中的使用指引；
- 决策机制在系统设计与工程实现中的一致性落地。

后续所有业务级应用、工程化实践与 AICoding 结构生成，均应以本部分所定义的业务关系为抽象依据，而不得反向修改方法论内核。
